



Progettazione e Implementazione di un Tool API-Agnostico per la Security Assurance di API REST

Laurea Magistrale in Sicurezza Informatica

Enea Manzi (65935A)

Relatore: Prof. Marco Anisetti

Correlatore: Prof. Claudio Agostino Ardagna

XX Luglio 2026



UNIVERSITÀ
DEGLI STUDI
DI MILANO



Contesto

- Architetture a **microservizi** cloud-native: la superficie di attacco cresce con la **proliferazione di endpoint**, spesso priva di governance sistematica
- Il **gateway API** come punto di enforcement centralizzato: autenticazione, autorizzazione, rate limiting
- Le vulnerabilità più significative sono **semantiche**, non sintattiche
 - 4 delle prime 5 categorie OWASP API Security Top 10 2023



Stato dell'arte e gap

- **G1 Cecità semantica:** scanner e fuzzer operano sulla **sintassi** HTTP, non sulla **semantica** dei contratti
- **G2 Portabilità/profondità:** strumenti specifici sono **approfonditi** ma legati a un **solo contesto**; quelli generici restano **superficiali**
- **G3 Riproducibilità:** assenza di **determinismo** preclude l'integrazione nei cicli di **sviluppo continuativo**
- **G4 Oracle problem:** nessun meccanismo sistematico per stabilire se un comportamento viola una **garanzia di sicurezza**



Obiettivi

Un tool **contract-driven** per la security assurance
di **qualsiasi API REST documentata**: agnostico, sistematico, riproducibile.

- **O1 Agnosticismo applicativo**: OAS e config.yaml come uniche sorgenti **di conoscenza** a runtime, nessun riferimento hardcoded al target
- **O2 Portabilità con profondità graduata**: il box gradient scala la profondità di assessment ai privilegi disponibili
- **O3 Riproducibilità deterministica**: output stabile e integrabile nei cicli di sviluppo continuativo
- **O4 Specified oracles**: criteri di valutazione affidabili derivati dalla conoscenza del dominio



Agnosticismo applicativo e contract-driven

- **Agnosticismo applicativo:** le uniche sorgenti di conoscenza del target a runtime sono `openapi.yaml` e `config.yaml`
 - nessun riferimento hardcoded al target nel codice
 - deployabile su **qualsiasi API REST documentata** senza modifiche
- **Paradigma contract-driven:** la specifica **OAS** guida la costruzione di probe **sintatticamente valide** e delimita la superficie di assessment



Tassonomia: otto domini di security assurance

- **Do API Discovery**
shadow API, inventario endpoint
- **D1 Authentication**
JWT, trasporto credenziali, revoca
- **D2 Authorization**
RBAC, BOLA, segregazione
- **D3 Data Integrity**
coerenza payload, HMAC
- **D4 Availability**
rate limiting, circuit breaker
- **D5 Observability**
audit logging, alerting
- **D6 Config & Hardening**
gateway, header HTTP
- **D7 Business Logic**
SSRF, race condition

18 test implementati, 14 pianificati, su 8 domini



Box gradient: profondità di assessment

Il livello di ogni test è una **conseguenza delle precondizioni di accesso disponibili**, non una tassonomia imposta dall'esterno.

BLACK_BOX

Nessun prerequisito

Simulazione attaccante esterno
senza credenziali; controlli
perimetrali del gateway

GREY_BOX

Token per ≥ 2 ruoli

Garanzie che presuppongono
stato autenticato; difetti RBAC,
BOLA, segregazione

WHITE_BOX

Admin API del gateway

Ispezione diretta della
configurazione tramite piano di
controllo



Specified oracles: il criterio di valutazione

Come si stabilisce se un comportamento osservato viola una garanzia di sicurezza?

- Generare input è risolto; **valutare il comportamento osservato** resta il collo di bottiglia del testing automatizzato
- APIGuard adotta **specified oracles**: criteri codificati **a priori** in ogni test, dalla conoscenza del dominio del controllo
- Tre fonti: schema **OAS**, standard tecnici, costruzione ad-hoc sul controllo specifico



Pipeline di esecuzione

- **Fasi 1-4 (bloccanti):** Inizializzazione, OpenAPI Discovery, Costruzione Contesti, Test Discovery + DAG
- **Fasi 5-7 (non bloccanti):** Esecuzione, Teardown LIFO, Report
- **Estensibilità:** connector gerarchici per tool esterni e discovery dinamico dei test, senza modifiche al core



Riproducibilità e integrazione CI/CD

- **DAG scheduler:** dipendenze dichiarate esplicitamente, risolte in ordine **topologico** — i prerequisiti vengono sempre prima
- **Config-driven:** **config.yaml** come unica fonte di verità per tutti i parametri operativi
- **Exit code semantici:** 0 CLEAN, 1 FAIL, 2 ERROR, 10 INFRA — segnale **machine-actionable** per CI/CD



Validazione sperimentale

Testbed

- **Forgejo** 14.0.3 esposto tramite **Kong**
3.9 DB-less, Docker locale
- Nessun riferimento a Forgejo nel
codice: conferma pratica
dell'agnosticismo

Risultati

Test attivi	18 su 7 domini
PASS	9
FAIL	7
SKIP	2
ERROR	0
Finding totali	98

Discrepanza rilevata

Forgejo dichiara autenticazione globale; alcuni endpoint pubblici rispondono senza credenziali — il tool ha distinto specifica da implementazione.

Idempotenza: 2 run indipendenti, risultati **byte-identici**, Δ wall-clock $< 0.3\%$



Conclusioni

Da **osservazione puntuale** a **garanzia misurabile**

- Tassonomia a **8 domini**, applicabile indipendentemente dall'implementazione
- Metodologia sistematica per gli **specified oracles** nel security testing REST
- Assessment engine deterministico a **DAG**, verificato empiricamente
- Validato su un target reale: **Forgejo + Kong**, non solo scenari sintetici



Sviluppi futuri

Estensioni operative

- Adapter per **gateway cloud managed** (AWS API Gateway, Azure APIM): oggi privi di Admin API accessibile, copertura limitata a BB e GB
- Completamento dominio **D5**
Observability

Direzioni di ricerca

- Piattaforma modulare per **protocolli eterogenei** (gRPC, GraphQL)
- **Formalizzazione** degli oracoli di sicurezza
- **Coverage augmentation** progressiva basata sui risultati precedenti
- **Parallelizzazione** intra-batch per ridurre il wall-clock



Progettazione e Implementazione di un Tool API-Agnostico per la Security Assurance di API REST

Grazie per l'ascolto! Qualche domanda?

Enea Manzi: enea.manzi@studenti.unimi.it



Limite strutturale del fuzzing sintattico

Via A

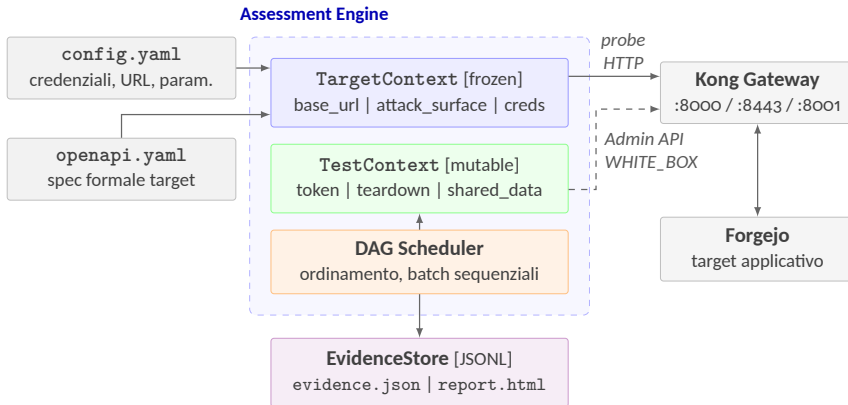


Via B



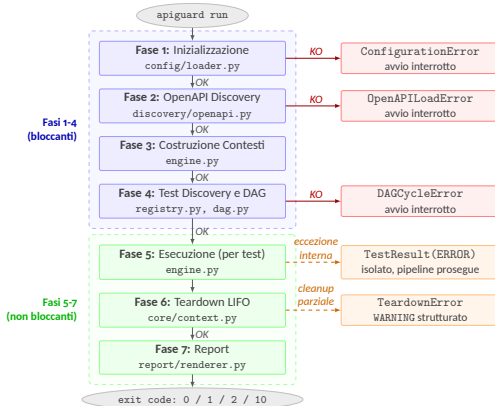


Architettura di APIGuard Assurance



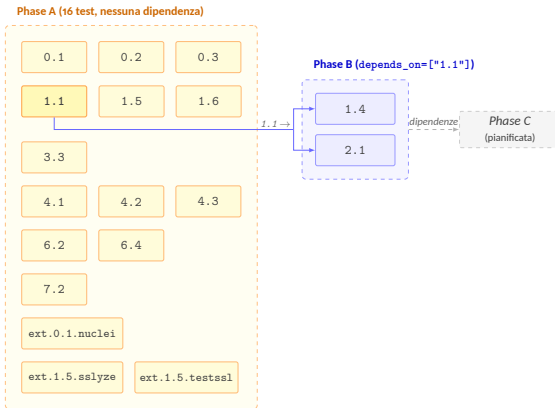


Pipeline a sette fasi





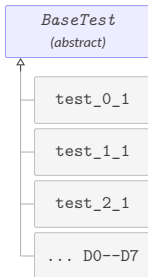
Topologia del DAG



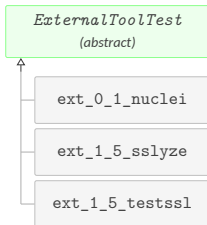


Gerarchia delle classi

Test nativi (DO-D7)

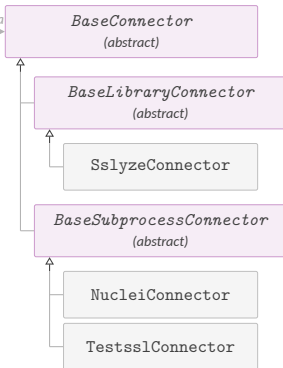


Test esterni



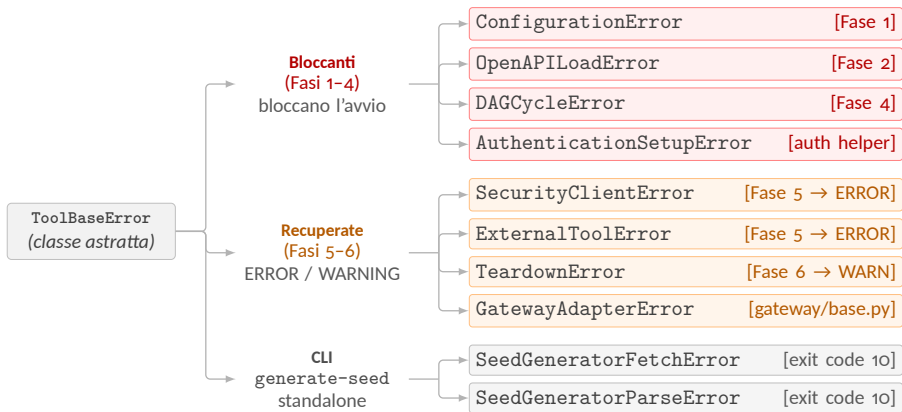
usa

Connettori





Gerarchia delle eccezioni





Struttura interna del TestContext

TestContext [mutable]: tre canali tipizzati con responsabilità distinte

Canale Token	Canale Teardown [LIFO]	Canale Shared Data
ROLE_ADMIN	push(fn_C)	"1.1.token"
ROLE_USER_A	push(fn_B)	"1.1.repos"
ROLE_USER_B	push(fn_A)	"ext.0.1.hits"
set_token(role, tok) get_token(role) → tok has_token(role) → bool	push_teardown(callable) drain order: fn_A → fn_B → fn_C	set_shared("id.k", v) get_shared("id.k") → v
Fase 5: test autenticati scrivono e leggono	Fase 5: push Fase 6: drain inv.	Fase 5: produttore scrive batch successivi leggono

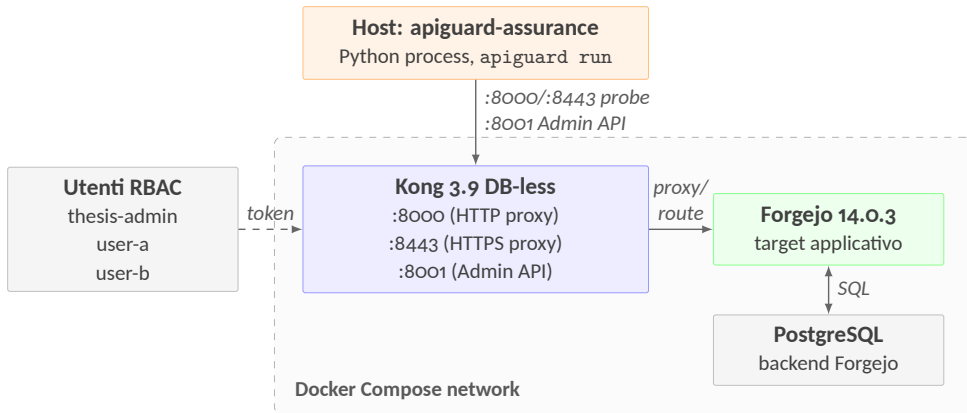


Mappa di copertura



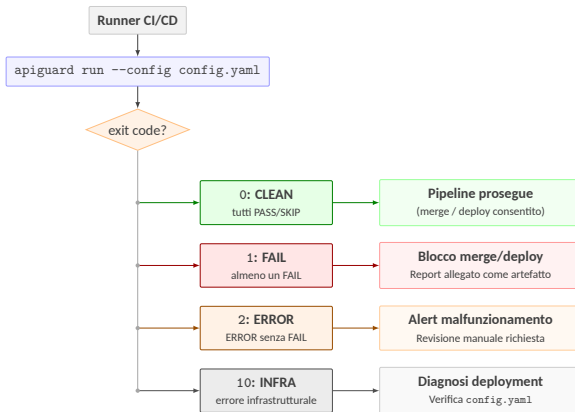


Topologia del testbed



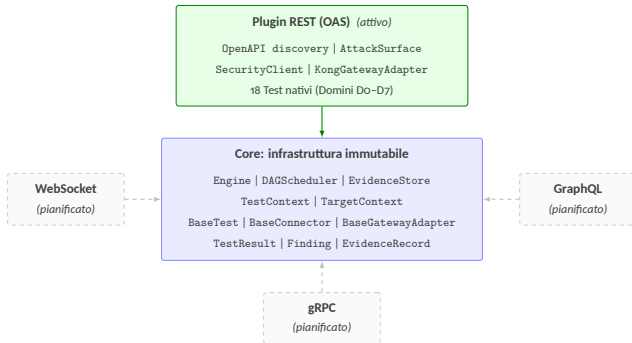


Routing degli exit code





Verso una piattaforma modulare

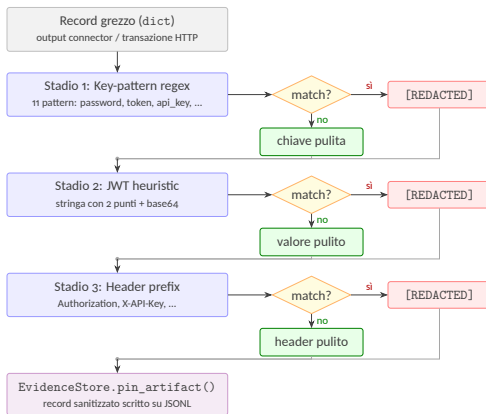


freccie continue: modulo attivo frecce tratteggiate: moduli pianificati

Il discovery dinamico via `pkgutil` permette il caricamento trasparente di moduli di terze parti

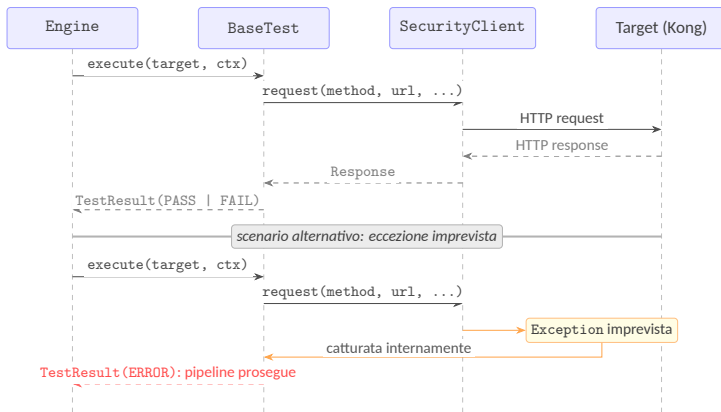


Sanitizzazione a imbuto dell'evidenza





Error isolation in Fase 5





Domini di Assurance

Dominio	Titolo	Ambito di verifica
D0	API Discovery	Inventario degli endpoint esposti; rilevazione di Shadow API non documentate nella specifica.
D1	Identity & Authentication	Meccanismi di riconoscimento del chiamante; robustezza del trasporto delle credenziali; ciclo di vita e revoca dei JWT.
D2	Authorization	Logiche di controllo degli accessi; difetti di segregazione orizzontale e verticale (RBAC, BOLA).
D3	Data Integrity	Coerenza strutturale dei payload; implementazione delle firme crittografiche (HMAC).
D4	Availability & Resilience	Difese contro l'esaurimento delle risorse; audit di circuit breaker e soglie di rate limiting.
D5	Observability	Qualità e tempestività della telemetria generata dal sistema.
D6	Configuration & Hardening	Rafforzamento della configurazione del gateway; policy di routing; header di sicurezza HTTP.
D7	Business Logic & Sensitive Flows	Falle semantiche complesse: elusione dei vincoli di processo, SSRF, race condition su operazioni critiche.



Box Gradient

Strategia	Prerequisiti	Razionale
BLACK_BOX	Nessuno (solo accesso di rete)	Controlli perimetrali applicati dal gateway a qualsiasi interlocutore; simulazione di un attaccante esterno senza credenziali.
GREY_BOX	Token per ≥ 2 ruoli distinti	Garanzie che presuppongono stato autenticato con identità di ruolo distinte; inaccessibili senza credenziali valide.
WHITE_BOX	Accesso Admin API del gateway	Configurazione statica verificabile per ispezione diretta tramite piano di controllo del gateway.



Risultati per test

Test ID	Dominio	Tipo	Status	Finding
0.1	Do: Discovery	Nativo	FAIL	16
0.2	Do: Discovery	Nativo	FAIL	1
0.3	Do: Discovery	Nativo	SKIP	0
1.1	D1: Auth	Nativo	FAIL	74
1.4	D1: Auth	Nativo	PASS	0
1.5	D1: Auth	Nativo	PASS	0
1.6	D1: Auth	Nativo	SKIP	0
2.1	D2: Authorization	Nativo	PASS	0
3.3	D3: Data Integrity	Nativo	PASS	0
4.1	D4: Availability	Nativo	FAIL	2
4.2	D4: Availability	Nativo	PASS	0
4.3	D4: Availability	Nativo	PASS	0
6.2	D6: Hardening	Nativo	PASS	0
6.4	D6: Hardening	Nativo	PASS	0
7.2	D7: Business Logic	Nativo	FAIL	1
ext.0.1.nuclei	Do: Discovery	Esterno	PASS	0
ext.1.5.sslyze	D1: Auth	Esterno	FAIL	1
ext.1.5.testssl	D1: Auth	Esterno	FAIL	3
Totale				98



KPI di idempotenza

Key Performance Indicator	Run 1	Run 2	Δ
PASS / FAIL / SKIP / ERROR	9 / 7 / 2 / 0	9 / 7 / 2 / 0	identico
Finding count totale	98	98	identico
pass_rate_pct	56.2%	56.2%	identico
assessment_duration_seconds	290.14 s	290.81 s	+0.23%
Exit code / label	1 / FAIL	1 / FAIL	identico



Performance baseline

Metrica	Run 1	Run 2
Wall-clock elapsed	290.14 s (4:50.14)	290.81 s (4:50.81)
User time	35.09 s	34.67 s
System time	41.51 s	41.62 s
Peak resident set size	287 MB	295 MB



Analisi statica

Tool	Esito	Note
ruff	0 errori	Ruleset E/W/F/I/N/UP/B/S/ANN
mypy	0 errori	Modalità strict: no-implicit-optional, disallow-untyped-defs, warn-return-any
bandit	0 finding severity $\geq$ medium	3 Low (B404, S603x2) soppressi con # noqa documentati: invocazione subprocess controllata per design
vulture	0 finding	Confidence $\geq$ 80%; metodi a dispatching dinamico esclusi via whitelist
pip-audit	0 CVE	Nessuna vulnerabilità nota nelle dipendenze dichiarate



Topologia del DAG (assessment)

Fase DAG	Cardinalità	Test
Phase A (nessuna dipendenza)	16 test	0.1 · 0.2 · 0.3 · 1.1 · 1.5 · 1.6 · 3.3 · 4.1 · 4.2 · 4.3 · 6.2 · 6.4 · 7.2 · ext.0.1.nuclei · ext.1.5.testssl · ext.1.5.sslyze
Phase B (depends_on = ["1.1"])	2 test	1.4 · 2.1



Verdetto di release

Aspetto	Stato	Dettaglio
Analisi statica	✓	0 errori su tutti e cinque gli strumenti
Type safety	✓	0 errori mypy strict; validazione Pydantic attiva su tutti i confini
Documentazione interna	✓	292/292 simboli pubblici con docstring (100%)
Riproducibilità della build	✓	SHA-256 identico su due build consecutive (451 261 byte)
Portabilità	✓	Cold install in ambiente pulito: <code>apiguard version</code> → 0.1.0



Versioni del testbed

Componente	Versione	Ruolo nel testbed
apiguard-assurance	0.1.0	Tool sotto validazione
Python	3.12.3	Interprete del tool
Forgejo	14.0.3 (gitea-1.22.0)	Target applicativo: API REST con specifica OpenAPI nativa
PostgreSQL	15-alpine	Backend di persistenza di Forgejo (dipendenza funzionale, non componente autonomo)
Kong	3.9 DB-less	API Gateway: proxy HTTP/HTTPS, instradamento verso Forgejo, Admin API su porta 8001
nuclei	3.8.0	Tool esterno per Shadow API discovery (test ext . 0 . 1)
nuclei-templates	10.4.3	Base di template di scansione versioned: garantisce che lo stesso set di firme venga usato in ogni run
testssl.sh	3.2.3	Analisi TLS black-box (test ext . 1 . 5 . testssl)
sslyze	(libreria Python)	Analisi TLS via libreria Python (test ext . 1 . 5 . sslyze)



Modelli di esposizione gateway

Modello	Traffico presidiato	Enforcement policy	Accesso al piano di configurazione	Verifica WHITE_BOX
Gateway API Dedicato (es. Kong, Apigee)	Nord-sud	Centralizzato (singolo strato)	Admin API diretta	Nativa
Kubernetes Ingress / Gateway API	Nord-sud	Centralizzato (via K8s API)	kubectl / Kubernetes API	Controller-dipendente
Service Mesh (es. Istio)	Est-ovest	Distribuito per sidecar proxy	Control plane (istiod)	N/A (est-ovest)
Gateway Managed Cloud (AWS, Azure, GCP)	Nord-sud	Centralizzato dal provider	SDK / API del cloud provider	Adapter dedicato